MediCore · Registro de hallazgos y verificación manual

Registro de los errores que aparecieron al construir cada entrega, su causa, la corrección y cómo se comprobó. Es el punto de partida del registro formal de defectos (entregable de la entrega 14: pasos, esperado, obtenido, severidad, prioridad y evidencia).

Tipos: **Producto** (defecto del sistema), **Prueba** (defecto de la prueba o de su dato), **Entorno** (herramienta o infraestructura local).

Entrega 4 · disponibilidad, agenda y citas

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H4-01	Producto	Con 12 reservas simultáneas al mismo horario, 11 respondían 500 en lugar de 409 (`40P01: deadlock detected`).	Las restricciones de exclusión de PostgreSQL insertan la fila y luego buscan conflictos; dos inserciones en conflicto se ven mutuamente sin confirmar y se bloquean entre sí.	La alta se serializa con `pg_advisory_xact_lock` por horario del médico y por paciente dentro de una transacción, y se reintenta (hasta 3 veces) ante 40P01/40001. Las restricciones siguen siendo la garantía final.	Las pruebas de concurrencia pasan: 1 reserva y 11 conflictos limpios. Repetidas 3 veces seguidas sin fallos.
H4-02	Producto	`GET /api/health` y las rutas protegidas respondían 500 en pruebas sin base de datos (3 pruebas fallaron).	`PatientAuthorizationHandler` recibía `MediCoreDbContext` por constructor y ASP.NET lo instancia en cada autorización; sin cadena de conexión, crear el contexto lanzaba excepción.	El manejador recibe `IServiceProvider` y obtiene el contexto solo cuando evalúa al médico.	`BasicEndpointTests` pasan; la API vuelve a arrancar y responder sin PostgreSQL configurado.
H4-03	Producto (web)	Toda página con `@section Scripts` (Agenda, Mi agenda, detalle de cita) daba 500: "sections defined but not rendered".	`_Shell.cshtml` es un layout anidado y no reenviaba la sección al layout base.	`_Shell` declara `@section Scripts` y la reenvía con `RenderSectionAsync`.	Agenda, Mi agenda y detalle cargan; comprobado en navegador.
H4-04	Producto (web)	En "Reservar cita", tras cambiar paciente o fecha, **Confirmar cita** no reservaba y la página quedaba igual.	El formulario no tenía `action`: después de "Actualizar horarios" la URL conservaba `?handler=Actualizar` y el envío volvía a ese manejador.	`action` explícito a `/Agenda/Reservar`.	Ver H4-05.
H4-05	Producto (web)	Con el `action` explícito, el envío daba una página vacía (HTTP 400).	Al declarar `action` a mano el *tag helper* no genera el token antifalsificación.	`asp-antiforgery="true"` en el formulario.	La reserva se completa y redirige al detalle de la cita; comprobado en navegador (envío con JavaScript, ver más abajo).

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H4-06	Prueba	`Administrador_configura_y_consulta_el_horario…` fallaba aunque los valores eran iguales.	Comparaba tuplas que contenían un arreglo (se comparan por referencia).	Se comparan por separado el arreglo de días y el resto.	Pasa.
H4-07	Prueba	`No_se_puede_cancelar_una_cita_que_ya_paso` lanzaba `DbUpdateException` al preparar el dato.	Se insertaba un `DateTime` con `Kind=Utc` en una columna `timestamp without time zone` (Npgsql lo rechaza).	El asistente de pruebas fuerza `DateTimeKind.Unspecified`.	Pasa.
H4-08	Prueba	Una aserción de `Alta_demanda…` estaba mal escrita (expresión sin sentido) y habría dado un resultado engañoso.	Error al redactar la prueba.	Detectada al revisarla antes de depender de ella; reescrita con conteos explícitos (4 creadas, 20 conflictos, 20 lecturas 200).	Pasa y es legible.
H4-09	Producto (datos de desarrollo)	Advertencia de EF Core `10103` (`First` sin `OrderBy`) al sembrar las citas de demostración.	Consulta sin orden determinista en `ClinicalDemoSeeder`.	`OrderBy` por correo normalizado.	Sin advertencia.
H4-10	Entorno	Al empezar, `/api/health/ready` respondía `unavailable`; los contenedores estaban detenidos.	Docker Desktop reiniciado; no es un defecto del sistema.	`docker compose up -d`.	PostgreSQL, Redis y RabbitMQ saludables.
H4-11	Entorno	Capturas y clics del panel del navegador fallaban a veces ("Screenshot timed out").	El panel no terminaba de dibujar con la ventana en segundo plano.	Se reintentó y, cuando no hubo forma de hacer el clic, se envió el formulario con JavaScript (ver más abajo).	Limita lo que cuenta como interacción real; ver detalle.

| H4-12 | Producto (web) | Con Cypress, escribir la fecha en el filtro de la agenda no la aplicaba (la URL conservaba la fecha anterior). | El envío automático escuchaba solo `change`. Cypress emite `input` y `change` solo al perder el foco; además, en navegadores de escritorio un campo de fecha emite `change` en cada edición válida mientras se teclea (por ejemplo, al escribir el primer dígito del año), lo que podría recargar la página a medias. Esto último es el comportamiento conocido del navegador: **no lo reproduje con un teclado físico**. | `autosubmit.js` y `reservar.js` escuchan `input` y `change` de los campos de fecha, esperan 700 ms a que termine la escritura y envían al instante al salir del campo. Las listas se siguen enviando al instante. | Cypress 8/8 en Electron y en Chrome. | | H4-13 | Prueba | Todas las pruebas fallaban en el `before` con "`Cypress.env()` was removed in Cypress version 16". | Cypress 16 eliminó `Cypress.env()`. | Valores no sensibles con `expose` y la contraseña con `cy.env()`. | Pasa. | | H4-14 | Prueba | Al usar `clear()` en la fecha, la agenda recargaba con la fecha vacía (la API toma "hoy"). | Borrar un campo de fecha lo cambia, y el envío automático lo envía. Comportamiento esperado,

no un defecto. | Las pruebas escriben la nueva fecha sin borrar antes. | Pasa. | | H4-15 | Prueba y producto (límite) | Elegir el paciente justo después de cambiar la fecha fallaba: "this element is `disabled`" (`--submitting form--`). | El navegador estaba enviando la actualización de horarios. En uso real, un cambio hecho en los pocos cientos de milisegundos de una recarga en curso se pierde. | La prueba espera cada recarga (`cy.intercept` + `cy.wait`). El límite del producto se acepta y queda documentado. | Pasa. | | H4-16 | Entorno | Cambiar los `.js` no tenía efecto y la prueba seguía fallando igual. | Los archivos estáticos se publican con huella en la compilación (`MapStaticAssets`): hay que recompilar y reiniciar la web. | Compilar antes de probar (anotado en `CLAUDE.md`). | Una ejecución perdida. | | H4-17 | Herramienta | Las capturas de Cypress salían recortadas y con barra de desplazamiento horizontal. | La ventana de Electron era menor que el *viewport* configurado (1440). | *Viewport* 1280×800. El desbordamiento horizontal se descartó midiendo `scrollWidth == clientWidth` en Agenda, Reservar y Pacientes, en Electron y en Chrome. | Capturas correctas. |

Hallazgo de diseño aceptado (no es un error): el calendario del prototipo tenía la leyenda "Seguimiento" (color azul) que no corresponde a ningún estado del sistema; se retiró y solo queda "Confirmada".

Verificación en navegador de la entrega 4: qué fue interacción real y qué fue JavaScript

Las pruebas automatizadas (129: 128 superadas y 1 omitida) no usan navegador. La revisión manual en el navegador integrado se hizo así; **no sustituye** a Cypress.

Interacción real (herramienta de navegador: teclas, clics, formularios y capturas)

Comprobación	Cómo
Inicio de sesión de Recepción (Elena)	Campos con la herramienta de formularios y envío con la **tecla Enter**.
Pantalla WEB-02 (agenda) con captura de pantalla	Navegación y captura; coincide visualmente con el diseño.
Cambio de médico en el filtro de la agenda	Valor del `select` fijado con la herramienta de formularios; la **propia página** disparó su envío automático y mostró las 4 citas de la Dra. Méndez.
Reservar cita: elegir paciente	Igual que arriba; la página se recargó sola con los horarios.
Reservar cita: clic en el horario y en "Confirmar cita"	**Clics reales**, hechos en un intento anterior; ahí se descubrió el defecto H4-04. Después de la corrección, los clics fallaron por tiempo agotado del panel (H4-11).
Pantalla WEB-09 (disponibilidad) con captura	Captura; coincide con el diseño.
Lectura del texto de la agenda del médico y del listado de pacientes del médico	Lectura de la página (no cambia datos).

Ejecutado con JavaScript (código que inyecté en la página; no es un usuario haciendo clic)

Comprobación	Qué hice	Qué **no** queda probado
Reserva correcta de la cita de las 09:30	`form.requestSubmit(...)`	El clic en el botón.
Conflicto de horario (10:30 reservado por la API antes del envío)	`requestSubmit` y lectura del mensaje y del estado de los campos	El clic; el aspecto visual del mensaje.
Cancelación de la cita	`form.submit()`	El cuadro de confirmación "¿Confirmas que deseas cancelar esta cita?" **no se mostró ni se probó**; `submit()` lo omite.
Cierre de sesión y nuevo inicio de sesión (Lucía y administrador)	`submit()` / `requestSubmit()` con valores asignados por código	El flujo con teclado y clics.
Disponibilidad: hora final anterior a la inicial, guardado válido y restauración	Valores asignados por código y `requestSubmit`	Los clics y los campos de hora con el selector del navegador.

Verificación con Cypress, sin inyectar JavaScript (entrega 4, repetida y ampliada)

Suite: `tests/cypress/e2e/recepcion-reserva.cy.js` (8 pruebas). Todas las acciones del usuario van por la interfaz (escribir, elegir, pulsar); la API solo prepara y limpia datos y simula a *otra persona* que ocupa el horario en el paso 7. Resultado: **8 de 8 superadas**, en **Electron 146** (4 ejecuciones seguidas) y en **Chrome 155** (2 ejecuciones), todas sin intervención manual y repetibles (los datos de prueba se limpian y reutilizan).

Paso pedido	Prueba	Resultado	Qué se comprobó
1. Iniciar sesión como Recepción	1	✓	Escribir correo y contraseña, pulsar "Iniciar sesión"; aparece "Elena Vargas · Recepción".
2. Elegir médico, fecha, paciente y horario	2 y 3	✓	Elegir médico y fecha en la agenda (la página se actualiza sola), "Reservar cita", cambiar la fecha en el formulario y volver (los horarios se actualizan), elegir paciente y pulsar el horario 09:00.
3. Confirmar con el botón	2 y 3	✓	Botón "Confirmar cita →"; abre el detalle con aviso "Cita confirmada correctamente", paciente, médico, hora 09:00 y estado Confirmada.
4. Abrir "Ver detalle"	4	✓	Desde la agenda, "Ver detalle ↗" de la cita lleva a la misma cita.
5. Cancelar rechazando la confirmación	5a	✓	El cuadro pide exactamente "¿Confirmas que deseas cancelar esta cita?", se responde que no, **no se envía ninguna petición**, el estado sigue Confirmada y la cita sigue en la agenda.
5. Cancelar aceptando	5b	✓	Se envía la petición, aviso "La cita fue cancelada", estado Cancelada y desaparece el botón.
6. El horario vuelve a estar disponible	6	✓	En la agenda 09:00 muestra " + Horario disponible" sin el paciente, y al pulsarlo el formulario trae 09:00 habilitado y elegido.
7. Conflicto de horario conservando lo ingresado	7	✓	Otra persona reserva 10:00 mientras se llena el formulario; al confirmar aparece "Este horario ya está ocupado. Selecciona otro para continuar.", **se conservan paciente, médico y fecha**, el horario 10:00 queda deshabilitado y sin elegir, y se puede reservar 10:30 sin empezar de nuevo.

Paso pedido	Prueba	Resultado	Qué se comprobó
8. Cambiar de mes en el calendario	8	✓	"Mes siguiente" muestra el mes siguiente con el día 1 marcado, otra vez el siguiente, "Mes anterior" dos veces regresa, y elegir el día 15 cambia la fecha de la agenda.

Evidencia: `reports/cypress/resultados-electron.xml`, `reports/cypress/resultados-chrome.xml` y capturas `reports/cypress/screenshots/recepcion-reserva.cy.js/01…08` (de la ejecución en Chrome).

Límites de esta verificación (no se marca como probado):

- Cypress dispara eventos del navegador; **no es una persona con teclado y ratón físicos**. En particular, escribir la fecha con teclado real (H4-12) no se reprodujo.
- El cuadro `confirm()` lo atiende Cypress: se comprobó que la página lo pide con el texto correcto y que respeta la respuesta, no su aspecto.
- Ejecución *headless*; no se midió accesibilidad (Lighthouse/axe), teclado, lector de pantalla ni móvil.
- Solo cubre Recepción. Médico, Administrador, pantallas de pacientes y médicos, smoke general y regresión siguen pendientes.

Con esta suite quedan **sustituidas** las comprobaciones con JavaScript de reserva, conflicto y cancelación de la sección anterior; las de cierre de sesión, inicio de sesión del médico y del administrador y de la pantalla de disponibilidad siguen siendo solo las descritas arriba (JavaScript).

Lo que no se verificó en navegador

- Aún sin verificar con interacción real: cierre de sesión, inicio de sesión de Médico y Administrador, y la pantalla de disponibilidad (WEB-09), que solo se probaron con JavaScript.
- Navegación solo con teclado, lector de pantalla, vista móvil y pantallas pequeñas.
- La web con la sesión de Recepción tras cambiar la zona horaria o cerca de medianoche.

Falta ampliar Cypress (smoke, otros roles, regresión y más escenarios negativos) y la revisión de accesibilidad con Lighthouse o axe.

Entrega 5 · expediente, consultas y recetas

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-01	Entorno / proceso	Se indicó que el PR de la entrega 4 estaba fusionado, pero `origin/main` seguía en la fusión del PR #3 (`00e510d`).	La fusión no llegó a `origin/main` (no se verifica con `gh` ; se comprobó con `git ls-remote`).	La rama `feature/entrega-5-consultas-recetas` se creó **sobre** `feature/entrega-4-agenda-citas` (rama apilada) para no perder la entrega 4.	Al fusionar el PR de la entrega 4, el de la 5 mostrará solo sus cambios. **Pendiente de tu confirmación.**
H5-02	Prueba	Al ejecutar varias suites seguidas, las preparaciones por API fallaban con `429 Too Many Requests` y todo quedaba en 0 pruebas.	El límite de intentos de inicio de sesión (10 por minuto por IP) funcionó como debe; las suites hacen muchos inicios de sesión.	En Development el límite sube a 60 por minuto (`appsettings.Development.json`); en otros ambientes sigue en 10.	15/15 pruebas, 4 ejecuciones seguidas por navegador.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-03	Herramienta	La captura de pantalla de Electron se colgaba de forma intermitente (cerca de 2 de cada 6 ejecuciones) y hacía fallar pruebas funcionales que sí pasaban.	Electron está obsoleto en Cypress 16; el problema es de la captura, no de la aplicación.	Las capturas de evidencia se toman solo en navegadores reales (Chrome); en Electron se ejecutan todas las aserciones.	Sin fallos en 4 ejecuciones seguidas en Electron.
H5-04	Producto (web)	Abrir el expediente de un paciente sin relación mostraba una página en blanco (404 sin cuerpo).	Las respuestas vacías no tenían página.	Página `/Estado` con `UseStatusCodePagesWithReExecute`: mensaje en español y se conserva el código HTTP.	Cypress paso 8: estado 404 y "No encontramos lo que buscas", sin datos del paciente.
H5-05	Prueba	Riesgo de falso negativo en la prueba de bitácoras: si el capturador no recibiera nada, "no contiene el dato" pasaría igual.	—	Se agregó un control: se escribe un texto conocido en la bitácora y se exige que el capturador lo vea antes de comprobar que el contenido clínico no aparece.	Pasa y es significativa.
H5-06	Herramienta	Una edición de archivo fue rechazada ("modified since read") tras cambiar de rama.	El archivo cambió al cambiar de rama.	Se volvió a leer y a escribir.	Sin efecto en el resultado.

Decisiones y desviaciones del diseño en esta entrega (no son errores):

- **Varios medicamentos (WEB-14):** el diseño muestra un solo medicamento; se agregó " + Agregar medicamento" y "Quitar este medicamento" (hasta 10), cada uno con dosis, frecuencia, duración e indicaciones, como se pidió. Funciona sin JavaScript.
- **Cita atendida (WEB-13):** se agregó la lista "Cita atendida" para asociar la consulta a la cita (requisito); se propone la cita desde la que se abrió el expediente.
- **"Crear receta ↗"** en cada consulta del expediente (el diseño solo ofrece "Guardar y crear receta" al registrar).
- **Laboratorio:** el diseño dice "Sin resultados registrados"; como el módulo no existe, se indica que está **pendiente** y no se muestra nada.
- **Qué ve el médico:** en el expediente solo lo que él escribió, no las notas de otros médicos del mismo paciente (mínimo privilegio). Es una decisión de seguridad que puedes cambiar.

Verificación de la entrega 5

- **Cypress `medico-expediente.cy.js` (10 pruebas tras el ajuste del expediente compartido), sin inyectar JavaScript:** iniciar sesión como Médico; abrir el expediente desde la agenda; recorrer Consultas, Recetas y Laboratorio (pendiente); guardar una consulta incompleta (errores); registrar la consulta; crear una receta con dos medicamentos (agregar y quitar filas conservando lo escrito); ver el detalle; volver al expediente y abrir la receta desde su tarjeta; comprobar que la cita ya no admite otra consulta; y que un paciente sin relación devuelve 404. **10/10 y la regresión de Recepción 8/8, en Chrome 155 y Electron 146** (una ejecución por navegador tras el ajuste; en la primera ejecución la prueba 10 falló porque `cy.visit` rechaza el 403 de Acceso denegado: se corrigió la prueba con `failOnStatusCode: false`, no la aplicación). No se repitió a distintas horas del día; la independencia horaria se debe al diseño (cita pasada insertada directamente).

- **Regresión de Recepción `recepcion-reserva.cy.js` (8 pruebas):** 8/8 en ambos navegadores, tras los cambios de esta entrega.
- **Límites:** Cypress dispara eventos del navegador (no es una persona con teclado físico); `confirm()` no interviene en este recorrido; no se midió accesibilidad ni móvil; cada ejecución agrega una consulta y una receta de prueba (la información clínica no se borra) con un texto único.

Entrega 5 · ajustes posteriores (expediente compartido y pruebas independientes de la hora)

Decisión de producto (reemplaza la de la primera versión de la entrega 5): el expediente es **compartido entre médicos autorizados**.

- Un médico con relación de atención válida con el paciente (una cita no cancelada o una consulta registrada) **lee** las consultas y recetas escritas por otros médicos; cada registro indica su autor (`IsOwn`, `DoctorName`) y la lectura de lo ajeno se audita como `other_author`, sin contenido.
- **No puede modificar** registros de otro profesional: solo el autor crea recetas sobre su consulta, solo se registra la consulta de una cita propia y no existe operación de edición ni de borrado de consultas ni de recetas (PUT, PATCH y DELETE devuelven 405, también para el autor).
- Sin relación válida no hay acceso (404 y `clinical.access_denied`); al cancelarse la última cita sin consulta se pierde la lectura. Recepción, Administrador y Laboratorio mantienen sus restricciones; el paciente lee solo sus recetas, de cualquiera de sus médicos.
- Motivo y riesgo: la relación de atención es una barrera baja (basta una cita no cancelada, incluso futura, con ese médico). Con el expediente compartido, esa cita da lectura de todo el historial clínico del paciente. Se acepta por indicación del equipo; si se quisiera endurecer, bastaría exigir una cita ya atendida o una consulta, cambiando `PatientAccess.HasCareRelationship` (una sola definición).

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-07	Prueba	El spec de Cypress del médico dependía de la hora del día: necesitaba un horario libre **más tarde hoy** y fallaba en los últimos minutos del día.	Para registrar una consulta hace falta una cita de hoy o anterior; la API no permite reservar en el pasado, así que la prueba reservaba "hoy, más tarde".	Cita ya pasada creada con una tarea de Cypress directamente en la base de datos **local** (`db:createPastAppointment`; se niega a conectar con cualquier servidor que no sea local). No se debilita ninguna validación de la API.	La prueba no mira el reloj.
H5-08	Prueba	La prueba del médico **cambiaba temporalmente el horario de la Dra. Méndez** (un médico de uso normal) y lo restauraba al final.	Necesitaba que atendiera todo el día.	Dos médicos dedicados a Cypress, sembrados solo en Development (`e2e.medico` y `e2e.colega`, ver `E2eFixtureSeeder`). Ningún médico de uso normal se toca.	Sin cambios en agendas reales.
H5-09	Prueba	Cada ejecución del spec del médico dejaba una consulta y una receta nuevas.	La información clínica no se borra por la API.	Tarea `db:cleanE2eClinical` al empezar y al terminar: borra consultas, recetas y citas **solo** de los pacientes `CYP-E2E-*`.	Sin acumulación; el estado vacío del expediente ahora se comprueba.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-10	Producto (diseño de código)	Con el expediente compartido, `PrescriptionAuthorizationHandler` necesitaba la relación de atención y la consulta estaba escrita dentro de otro manejador.	Duplicación.	Servicio `ICareRelationship` (usado por ambos manejadores; mismo criterio que el listado de pacientes). Las pruebas unitarias lo simulan sin base de datos.	31 pruebas unitarias.

Entrega 6 · laboratorio y notificaciones asíncronas

Qué se implementó, qué se ejecutó y qué no

Implementado: resultados de laboratorio (dominio, migración `LaboratorioYNotificaciones`, `LabsController`, `LabResultAuthorizationHandler`), pantallas WEB-16/17/18 y la pestaña Laboratorio del expediente, outbox transaccional con publicador en segundo plano, consumidor de `LabResultReady` con reintentos, DLQ e idempotencia, API de notificaciones del paciente (para la futura app móvil), scripts de ejecución en contenedor Linux y de evidencia.

Ejecutado (todo con resultado real):

Qué	Dónde	Resultado
`dotnet test MediCore.slnx` (`scripts/dotnet-linux.sh test MediCore.slnx`)	contenedor Linux `dotnet/sdk:10.0`, Testcontainers con PostgreSQL 17, Redis 7 y RabbitMQ 4	**226/226**: 44 unitarias y 182 de integración (43 de `LaboratoryApiTests`, 8 de `MessagingTests`)
Cypress `laboratorio-resultados`, `medico-expediente`, `recepcion-reserva`	Chrome 155 y Electron 146, contra la API y la web en contenedores Linux conectados al compose real	**26/26** en cada navegador (8 + 10 + 8)
`scripts/evidencia_laboratorio.py`	RabbitMQ real de compose (se detiene y se vuelve a iniciar)	Escenarios A–D correctos; informe en `documentacion/evidencias/entrega-6-laboratorio.md`
Migración	generada en el contenedor y aplicada a la base de desarrollo al arrancar la API (solo agrega tablas y una columna; no borra datos)	aplicada sin errores

No ejecutado o no verificado: - Nada se ejecutó de forma nativa en Windows: **Smart App Control** (activo) bloquea los `.dll` sin firma recién compilados; no se desactivó (ver H6-01). El código se compila y prueba en Linux. - `dotnet format --verify-no-changes` en Linux reporta diferencias de espacios en archivos que esta entrega no tocó (por ejemplo `AgendaDay.cs`), así que no es concluyente; **no se afirma** que el formato esté verificado. - No se midió rendimiento (k6/JMeter pendiente), accesibilidad ni uso móvil; no hay Insomnia ni Maestro. - El caso "RabbitMQ detenido" no se automatiza en Cypress (no controla Docker); se cubre con `MessagingTests` y con el script de evidencia. - No se probó la pérdida definitiva de la conexión con el broker más allá de detenerlo y volver a iniciarlo, ni varias instancias de la API publicando a la vez (la consulta usa `FOR UPDATE SKIP LOCKED`, pero no hay prueba con dos procesos). - Cambio de `docker-compose.yml` (nombre de nodo fijo `medicore-rabbitmq`) **sin aplicar**: el contenedor de RabbitMQ en ejecución no

se recreó para no tocar los datos existentes; al recrearlo, RabbitMQ empieza con un almacén de colas vacío dentro del mismo volumen (los datos anteriores no se borran).

Decisiones y adaptaciones al diseño

- **Quién ve qué en laboratorio:** Laboratorio ve todos los resultados (equipo compartido; decisión revisable hacia "solo los propios"); el Médico, solo con relación válida de atención; el Paciente, solo los suyos; Recepción y Administrador, ninguno. Acceso ajeno → 404 (no confirma existencia) + `clinical.access_denied`; rol sin permiso → 403.
- **Resultados inmutables:** no hay edición ni borrado (PUT/PATCH/DELETE → 405); una corrección es un resultado nuevo, como las recetas.
- **WEB-17:** el diseño tiene un solo parámetro; se permiten de 1 a 10 (filas que se agregan y quitan sin perder lo escrito, también sin JavaScript). Unidad y rango son opcionales (hay resultados cualitativos). Se agrega un filtro para elegir al paciente. **No se copió** la casilla "Simular envío de notificación pendiente" del prototipo: la notificación pendiente es un estado real (broker caído o consumidor con fallos), no una simulación.
- **WEB-18:** "Reintentar notificación" aparece solo para Laboratorio y solo si la notificación sigue pendiente. El médico ve el detalle y vuelve a su expediente.
- **Estado de la notificación:** "Enviada" = existe la notificación interna del paciente; "Pendiente" = todavía no (el resultado ya está guardado).
- **El evento lleva solo identificadores**; el título y el cuerpo de la notificación (que incluye el tipo de examen) se arman al consumirlo y se guardan en la tabla `notifications`, que solo lee el propio paciente.
- **Alcance de mensajería:** solo `LabResultReady`; los demás eventos siguen en la entrega 7 (ver plan).

Hallazgos

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H 6-01	Entorno	Migraciones, pruebas y API dejaron de cargar los `.dll` del proyecto: "Una directiva de Control de aplicaciones bloqueó este archivo" (0x800711C7).	Smart App Control (estado activo) de Windows bloquea binarios sin firma recién compilados; antes de esta sesión no bloqueaba.	**No se desactivó** (es un ajuste de seguridad del equipo). Se añadieron `scripts/dotnet-linux.sh` y `scripts/dev-stack-linux.sh`: build, migraciones, pruebas y la pila de desarrollo corren en contenedores Linux del SDK oficial, sin tocar volúmenes ni redes de compose.	Todo lo ejecutado en esta entrega usó ese camino; falta repetirlo de forma nativa en un equipo sin el bloqueo.
H 6-02	Defecto de producto (detectado por `MessagingTests`)	Tras detener y volver a iniciar RabbitMQ, el consumidor dejaba de recibir mensajes y las notificaciones no se generaban.	El publicador, al ver la conexión cerrada mientras el cliente la recuperaba, creaba una conexión nueva y cerraba la anterior, que era la del consumidor.	`RabbitMqConnection` ya no reemplaza una conexión existente (se recupera sola, incluido el consumidor).	`Con_el_broker_detenido…` y las demás pruebas de mensajería pasan; la evidencia B lo repite contra el RabbitMQ real.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H 6-03	Prueba	Tres pruebas fallaban con `operator does not exist: jsonb ~~ jsonb`.	`Payload.Contains(...)` sobre una columna `jsonb` se traduce a `LIKE`.	Ayudante `OutboxIdsForLabResultAsync` con `Payload->>'labResultId'`.	43/43 de laboratorio.
H 6-04	Prueba	`BrokerFixture` fallaba con "ports are not available ... Intento de acceso a un socket no permitido".	El puerto fijo elegido caía en un rango reservado por Windows (Hyper-V).	Puerto fijo aleatorio con reintento ante ese error (hace falta puerto fijo para detener y reiniciar el broker con la misma dirección).	8/8 de mensajería.
H 6-05	Prueba	La validación de los atributos de un `record` posicional no se detectaba con `Validator.TryValidateObject`.	Los atributos están en los parámetros del constructor, no en propiedades; ASP.NET Core sí los aplica.	Ayudante que valida el valor de cada parámetro contra sus atributos.	12 pruebas unitarias de validación.
H 6-06	Prueba existente	El recorrido 2 de Cypress del médico esperaba el aviso "laboratorio: pendiente".	La pestaña Laboratorio ahora lista resultados.	Espera "Sin resultados registrados." y el spec y la limpieza (`db:cleanE2eClinical`) incluyen resultados, notificaciones y eventos de laboratorio de los pacientes `CYP-E2E-*`.	10/10 y regresión de Recepción 8/8.
H 6-07	Riesgo abierto	`dotnet format` no es concluyente en Linux; el consumidor se probó con un solo proceso; el almacén de colas de RabbitMQ cambia al recrear el contenedor con el nuevo nombre de nodo.	Ver "No ejecutado o no verificado".	Documentado; sin corrección en esta entrega.	Abierto.

Evidencia reproducible: `documentacion/evidencias/entrega-6-laboratorio.md` (generada por `python scripts/evidencia_laboratorio.py`) y `documentacion/evidencias/entrega-6-pruebas.md` (comandos y resultados de las ejecuciones).

Entrega 7 · app móvil del paciente

Qué se implementó, compiló, probó y qué no

Estado	Detalle
Implementado	MOB-01 a MOB-12 en `src/MediCore.Mobile` (XAML + Shell) sobre `src/MediCore.Mobile.Core` (cliente de la API con renovación de tokens, ViewModels con estados de carga/vacío/error/sesión vencida, reserva con clave de idempotencia y recuperación ante horario ocupado); sesión en `SecureStorage`; configuración de red solo-HTTPS con excepción para desarrollo; 8 flujos Maestro; scripts de preparación y ejecución; documentación en `documentacion/05-app-movil.md`.
Compilado	`MediCore.Mobile.Core` y las pruebas móviles (SDK de .NET en contenedor Linux). La app MAUI se compiló a APK de desarrollo (arm64-v8a y x86_64) en un contenedor Linux con el workload `maui-android` y la plataforma Android 36: **Build succeeded**. Compilar en Linux **no equivale** a probarla en Android.

Estado	Detalle
Probado (verificado)	50 pruebas unitarias de la lógica móvil (xUnit, API simulada solo para aislar el ViewModel) y 17 pruebas del cliente contra la **API real** (PostgreSQL y Redis reales): login y rechazo del personal, renovación y rotación de tokens, cierre de sesión que revoca, reserva y cancelación, idempotencia (5 reintentos simultáneos → 1 cita), horario ocupado (409), IDOR en citas/resultados/recetas/notificaciones, notificaciones. Los 6 escenarios de datos de Maestro se prepararon con éxito contra la API real.
Probado en Android	**Sí, en el emulador `medicore_pixel` (Android 35, WHPX) con Maestro 2.12.0: 8 de 8 flujos aprobados** contra la API real en Docker (`documentacion/evidencias/entrega-7-maestro.md`). No se probó en un dispositivo físico, ni accesibilidad, rendimiento o rotación.

Herramientas (instaladas con autorización)

SDK de Android (command-line tools, platform-tools, emulator y la imagen `android-35;google_apis;x86_64` desde dl.google.com, licencias aceptadas con tu autorización) y Maestro 2.12.0 (release oficial de GitHub). No había un SDK reutilizable (Visual Studio 18 Insiders no trae uno). WHPX estaba disponible: no hizo falta reiniciar ni habilitar virtualización. El workload `maui-android` no se instaló en Windows (se compila en contenedor Linux). Smart App Control sigue activo.

Hallazgos y decisiones

ID	Tipo	Qué ocurrió	Causa / decisión	Resultado
H7-01	Entorno	No hay workload, SDK, emulador ni Maestro.	No se instaló nada en el equipo sin tu aprobación; se separó la lógica en `Mobile.Core` para probarla ahora; la compilación Android se intentó en un contenedor Linux.	Ver el reporte.
H7-02	Prueba	El token de acceso no cambia si se renueva en el mismo segundo (la prueba exigía que fuera distinto).	El JWT es determinista para la misma sesión y segundo; lo que debe rotar es el refresco.	La prueba comprueba la rotación del refresco y que reutilizar el refresco viejo revoca la sesión.
H7-03	Decisión	Las notificaciones del diseño incluyen "Cita confirmada" y "Nueva receta".	El sistema solo genera `LabResultReady` (los demás eventos están pendientes en el plan); no se inventó backend nuevo.	La app muestra cualquier notificación y abre el contenido solo cuando existe (hoy, resultados).
H7-04	Riesgo (cerrado)	Los identificadores de Maestro dependían de cómo MAUI expone `AutomationId` y `SemanticProperties`.	Los flujos usan texto visible y descripciones de accesibilidad.	**Confirmado en emulador**: `Fecha de la cita` y `Notificaciones` funcionan como selectores.
H7-06	Defecto de código (detectado al compilar)	La primera compilación real de la app falló: `DatePicker.DateSelected` entrega `DateTime?` en MAUI 10 (CS1503); además `DisplayAlert` está obsoleto.	El código MAUI se escribió sin poder compilarlo hasta tener el workload en un contenedor.	`SlotsPage` usa `e.NewDate is { } date`; el diálogo usa `DisplayAlertAsync`.
H7-07	Entorno	La compilación fallaba con NETSDK1047 (sin destino `android-x64` / `android-arm64`) y el instalador del SDK con "UserName must have a valid value".	Falta `RuntimeIdentifiers` en el restore; el contenedor no define `USER`.	`RuntimeIdentifiers` en el `.csproj` y `USER` / `USERNAME` en `android-build-linux.sh`.

ID	Tipo	Qué ocurrió	Causa / decisión	Resultado
H7-05	Riesgo abierto	Smart App Control sigue activo; no se probó compilar la app MAUI de forma nativa en Windows.	No se desactivó; la compilación se hace en contenedor Linux y la prueba en el emulador de Windows (binarios de Google firmados).	Abierto solo para la compilación nativa.
H7-08	Defecto (solo visible en Android)	La app se cerraba al abrir: `No assemblies found ... Fast Deployment`.	El APK Debug no incluía los ensamblados.	`EmbedAssembliesIntoApk=true`, `AndroidUseSharedRuntime=false` (APK ≈84 MB).
H7-09	Defecto (solo visible en Android)	Tras lo anterior: `lib/x86_64 ... MUST be STORED`.	Bibliotecas nativas comprimidas en el APK.	`android:extractNativeLibs="true"` en el manifiesto.
H7-10	Entorno	`adb install -r` fallaba con `INSTALL_FAILED_UPDATE_INCOMPATIBLE` y se seguía probando una versión vieja (parecía que los arreglos no funcionaban).	Cada compilación en contenedor genera otra clave de depuración.	Desinstalar antes de instalar (documentado).
H7-11	Prueba	En la primera pasada fallaron los flujos 01, 02 y 07.	01: el texto de la tarjeta lleva un ícono delante; 02: se intentaba tocar un campo ya escrito; 07: el emulador quedó `offline` un instante (no la app).	Selectores con expresión regular (01), formulario reiniciado (02), repetición (07).
H7-12	Herramienta	`maestro_run.py` no encontraba `maestro` en Windows y los subflujos exigían cabecera de configuración.	`maestro` es un `.bat`; Maestro exige `appId` y `---` también en subflujos.	`shutil.which` y cabecera en `subflows/login.yaml`.

Mejora pendiente (H7-13): clave de firma de depuración estable. Cada compilación en contenedor crea otra clave, por lo que hay que desinstalar la app antes de instalar un APK nuevo y se pierden sus datos locales (la sesión guardada). Pendiente: generar una vez un `debug.keystore` propio **fuera del repositorio** (por ejemplo en un volumen de Docker o en la carpeta de usuario), montarlo en `scripts/android-build-linux.sh` y configurar la firma (`AndroidSigningKeyStore`) para actualizar con `adb install -r`. No se versiona nunca la clave.

Evidencia: `documentacion/evidencias/entrega-7-pruebas.md`.

Web de extremo a extremo (2026-11-02) · Acceso, Recepción, Administrador, Médico y Laboratorio

Prioridad cambiada: se pausó Azure/Terraform/Ansible/Jenkins y la app móvil (sin borrar nada) para revisar la web del personal. Rama `feature/web-extremo-a-extremo` (desde `main`; la entrega 8 sigue sin fusionar).

Qué se ejecutó (resultados reales)

- **Cypress en Chrome 155: 102/102** (10 specs, ver `tests/cypress/README.md`) contra la API y la web en contenedores (`scripts/dev-stack-linux.sh`). No se re-ejecutó en Electron.
- **xUnit: 293/293** (44 + 50 + 199) con `scripts/dotnet-linux.sh test MediCore.slnx`.
- **Rastreo por rol** (`scripts/web_crawl.py`, solo GET): las 18 pantallas del diseño existen y responden; permisos por rol correctos; sin textos de "pendiente".
- **Caída de la API** (`scripts/web_conexion.py`, a mano, no Cypress): todas las pantallas principales responden 200 con un mensaje en español ("No se pudo cargar…"), sin trazas; la API se volvió a levantar.

Hallazgos

ID	Tipo	Qué ocurrió	Resultado
HW-01	Defecto	La página de inicio decía que las pantallas llegarían en "próximas entregas".	Accesos directos por rol. Corregido.
HW-02	Defecto	`max` de la fecha de nacimiento tenía un formato inválido y el navegador no limitaba fechas futuras.	Corregido (`yyyy-MM-dd`).
HW-03	Defecto (visible en celular)	Con el menú del Administrador (más enlaces) la página se ensanchaba a 427 px en una pantalla de 360 px.	`grid-template-columns: minmax(0,1fr)` en `app.css`; el menú se desplaza en su franja. Corregido.
HW-04	Defecto (visible en celular)	Las tablas anchas (médicos, usuarios, pacientes, resultados) podían ensanchar la página.	Se desplazan dentro de su panel. Corregido.
HW-05	Prueba	Tras elegir paciente en Reservar la página se recarga sola; la prueba actuaba antes de la recarga.	Las pruebas esperan cada recarga. No era defecto de la web.
HW-06	Prueba	La notificación del laboratorio es asíncrona: el estado "Enviada" no es inmediato.	La prueba acepta Enviada o Pendiente en el listado.
HW-07	Prueba	La tarea `db:createPendingLabResult` buscaba el correo en mayúsculas; el normalizado está en minúsculas.	Corregida.

Pendiente / no comprobado

- Auditoría de accesibilidad automatizada (axe/Lighthouse) y lector de pantalla: solo se comprobaron etiquetas, orden de teclado y desbordes con pruebas propias.
- La concurrencia real de doble reserva se cubre con xUnit/API (la prueba Cypress reserva dos veces seguidas, no en paralelo).
- Electron tras la ampliación; Firefox/Safari; navegadores móviles reales (solo viewport de 375 px).
- Insomnia, k6/JMeter, Maestro adicional, Jenkins/DevSecOps y Azure siguen pendientes (pausados por decisión).

Entrega 8 · contenedores e infraestructura

Qué se implementó, ejecutó y qué no

- **Implementado:** 4 Dockerfile (api, web, worker, migrator), proyecto `MediCore.Worker`, `docker-compose.qa.yml` sobre el Compose existente, proxy Caddy, scripts `qa.sh` / `qa-smoke.sh` / `qa-persistence.sh` / `validate-iac.sh`, Terraform (bootstrap del estado + VM de QA), Ansible (base, docker, aplicación), `documentacion/06-infraestructura.md`.
- **Ejecutado:** imágenes construidas; ambiente QA levantado desde cero; humo de web y API; persistencia tras `restart` y `down` + `up` (13/13); puertos de datos no publicados; Cypress de Recepción 8/8 a través del proxy; APK contra el ambiente en contenedores y 8/8 flujos Maestro; xUnit 293/293 tras los cambios; validación de Terraform, Ansible, hadolint y compose. Detalle: `documentacion/evidencias/entrega-8-verificacion.md`.
- **No ejecutado:** `terraform plan/apply`, `ansible-playbook` contra un servidor, HTTPS real, prueba en Azure. **La validación de archivos no se presenta como un despliegue.**

Hallazgos

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H8-01	Defecto (solo visible en contenedor)	La web quedó `unhealthy`: `UnauthorizedAccessException` al escribir en `/keys`.	El volumen nuevo es propiedad de root y la imagen corre sin privilegios.	`Dockerfile` de la web crea `/keys` con el dueño del usuario de la aplicación.	Web saludable; las claves persisten.
H8-02	Defecto (solo visible en una base nueva)	`--seed-qa` fallaba: `ClinicalDemoSeeder` no encontraba a la Dra. Méndez.	`DevelopmentBootstrap` sembraba primero los médicos de las pruebas E2E y el sembrado de demostración solo crea médicos si no hay ninguno; en desarrollo no se notaba porque ya había datos.	Se siembra primero la demostración y después los datos E2E.	Base nueva con 8 usuarios, 5 médicos y 5 pacientes.
H8-03	Prueba	Con el contenedor `-p medicore` el `name:` del archivo base podía colisionar con los volúmenes de desarrollo.	Dos archivos Compose definen `name`.	`docker-compose.qa.yml` fija `name: medicore-qa` y `scripts/qa.sh` siempre usa `-p medicore-qa`; se comprobó que los volúmenes de desarrollo no cambian.	Sin efectos en el desarrollo.
H8-04	Herramienta	`ansible-lint` marcó 2 faltas (nombre en minúscula, variable de rol sin prefijo) y `apt_repository` está obsoleto.	Primer borrador del playbook.	Corregidas; repositorio con `deb822_repository`.	Perfil *production* sin fallos.
H8-05	Prueba	En la primera pasada de Maestro contra QA fallaron 01 y 03 en 8-9 s.	`DeviceServerDiedException` del driver de Maestro en el emulador; no la app.	Se repitió la suite completa.	8/8.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H8-06	Seguridad (decisión)	Detrás del proxy, el límite de intentos de login por IP veía siempre la IP del proxy.	Faltaban cabeceras reenviadas.	`ForwardedHeaders__Enabled` (solo en el Compose de QA, donde la API no publica puertos).	Documentado; IP real del cliente.
H8-07	Riesgo abierto	La construcción de imágenes ocurre en la VM (no hay registro) y requiere acceso al repositorio.	Evitar el costo de un registro en esta etapa.	Anotado: GHCR/ACR con el pipeline de Jenkins.	Abierto.
H8-08	Riesgo abierto	Copias de seguridad de PostgreSQL y monitoreo no existen; el disco de datos es la única protección.	Fuera del alcance de esta entrega.	Resuelto: copias cifradas automáticas y restauración probada, ver `evidencias/respaldos.md` . El monitoreo sigue pendiente.	Cerrado.
H8-09	Riesgo abierto	No hay prueba xUnit para el worker ni para `--seed-qa` (solo verificación con el ambiente).	Se verificó con el ambiente en contenedores.	Anotado.	Abierto.

Evidencia: `documentacion/evidencias/entrega-8-verificacion.md` y `entrega-8-maestro-qa.md`.

Cierre · rendimiento, seguridad y pipeline (2026-11-06)

El registro formal, con pasos, severidad y prioridad, está en 12-registro-de-defectos.md.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
HP-01	Prueba	Stress y Spike aprobaron casi sin reservas.	Load había llenado las agendas.	Cada corrida crea sus médicos, y falla si no reserva.	Corridas válidas.
HP-02	Producto	Quedaron 14 405 eventos pendientes después del pico.	El outbox publicaba 20 eventos cada 2 s.	Lote de 200 y drenado continuo.	0 pendientes.
HS-01	Producto	SonarQube encontró bugs S1751.	Bucles que salían en la primera vuelta.	Lectura corregida.	0 bugs.
HS-02	Producto	La puerta rechazó el código nuevo.	Duplicación entre web y móvil, sin pruebas.	`ProblemDetailsReader` compartido y 4 pruebas.	Puerta aprobada.
HS-03	Prueba	La puerta por defecto aprobaba sin condiciones.	Era un proyecto sin puerta asignada.	Puerta propia "MediCore".	La puerta exige condiciones reales.
HA-01	Producto	axe encontró contraste insuficiente en el calendario.	Colores del diseño.	Colores en `app.css` .	0 infracciones.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
HZ-01	Producto	Un carácter nulo provocaba un error 500.	PostgreSQL rechaza NUL en texto.	`InvalidTextExceptionHandler` responde 400.	Prueba de regresión.
HZ-02	Producto	ZAP reportó cabeceras faltantes.	No estaban en Caddy.	Agregadas.	0 fallas en ZAP.
HZ-04	Prueba	ZAP API recibía casi solo 401.	Probaba cerrar sesión y perdía la sesión.	Rutas de sesión excluidas.	Análisis con sesión; se completó en HZ-05.
HZ-05	Prueba	ZAP no llegaba a la lógica clínica.	Usaba solo Administrador, identificadores inventados y cuerpos que la API rechazaba.	Análisis por rol, con identificadores y cuerpos válidos. El orden importa: el análisis activo altera datos.	40 de 45 operaciones alcanzadas, 0 errores 500.
HC-01	Producto	7 archivos fallaban `dotnet format` en Jenkins.	La validación nunca se había ejecutado.	Formato aplicado.	Etapa 3 aprobada.
HJ-01	Entorno	Jenkins no tenía `docker`.	Debian separa el cliente en `docker-cli`.	Paquete correcto.	Corre etapas.
HJ-02	Entorno	El smoke fallaba en Jenkins.	`127.0.0.1` era el propio contenedor.	Jenkins entra a la red del proxy.	Etapa 10 aprobada.
HJ-03	Prueba	Insomnia falló en Jenkins sin horarios libres.	QA acumuló 761 médicos y 34 655 citas de k6 y ZAP.	QA se recrea en cada corrida y la fecha varía.	65 de 65.
HD-01	Herramienta	Dependency-Check reportó 4 críticas.	Confundió bibliotecas cliente con servidores.	Revisadas y aceptadas con motivo.	Ver reporte de seguridad.
H9-01	Producto	QA no arrancaba tras reiniciar Docker.	Jenkins crea los contenedores con rutas de su workspace.	Archivos dentro de las imágenes.	`qa.sh smoke` correcto.
H9-02	Producto	Ansible fallaba en un Ubuntu mínimo.	Faltaba `python3-debian`.	Paquete agregado.	Dos pasadas sin fallas.
H9-03	Entorno	Los 8 flujos de Maestro fallaron a la vez.	La cortina de notificaciones del emulador tapaba la app.	Cortina cerrada antes de ejecutar.	8 de 8.
HJ-04	Entorno	Jenkins perdió su historial al reiniciar Docker.	`/var/jenkins_home` vive en la VM de Docker Desktop.	Aceptado; las consolas están en `reports/jenkins/`.	Numeración reiniciada.
HE-01	Error propio	Se borraron capturas antiguas de Cypress que no estaban en Git (unos 9,7 MB).	Una limpieza con `rm -rf`.	No se pueden recuperar. Cypress genera capturas nuevas si una prueba falla.	Sin efecto en las pruebas.

Entregas anteriores

Los hallazgos de las entregas 0 a 3 se trasladarán al registro formal de defectos en la entrega 14. Los principales: login con 503 por usar `localhost` para Redis (el cliente probaba IPv6 y Docker publica solo IPv4; corregido con `127.0.0.1`), una prueba que comparaba el `traceId` de dos respuestas de error, una prueba del catálogo de médicos afectada por la caché compartida con otras pruebas (se descarta la clave antes de insertar datos directos) y la interfaz de `IAsyncLifetime` de xUnit v2.